SRC/FEATURE.PY

- My Path is absolute.
- Assigned my 7 columns to a vari(SERVICE_COL) for those columns where "No internet service" and "No phone service" are "No".
- LOAD_RAW()
- To load my raw data with path with level 2 pointing to my telco df
- TotalCharges is stored as text(NaN) and is blank for customers with tenure 0.
- Filling missing values with 0
- ENGINEER()
- df.copy() so my original df stays untouched. No side effects.
- Looping through SERVICE_COL and replacing "No internet service"/"No phone service" with "No". 3 categories become 2.
- num_addons = counting how many paid services each customer has (0 to 6). Dropped MultipleLines from the count, it is phone not internet addon.
- avg_monthly_spend = TotalCharges / tenure. Separates old customer paying $80 from new customer paying $80. np.where guards tenure 0, falls back to MonthlyCharges.
- charge_delta = MonthlyCharges - avg_monthly_spend. Positive means recent price hike.
- tenure_bucket = pd.cut into 5 bands. Churn risk is front loaded, not linear. Bin starts at -1 so tenure 0 is included. np.inf at the end catches everything above 48.
- is_month_to_month and is_electronic_check = my 2 strongest categorical signals, made explicit as 1/0.
- BUILD_MATRIX()
- y = my target. "Yes"/"No" becomes 1/0.
- X = everything else. Dropped TARGET so model cannot see the answer (leakage). Dropped ID_COL because it is unique per row, would create 7043 columns and model would memorise individuals instead of learning patterns.
- cat_cols = auto finding my text columns with select_dtypes(object).
- get_dummies with drop_first=True for one hot encoding. Drops one category per feature to avoid the dummy variable trap.
- Cleaning column names. XGBoost and SHAP reject spaces and brackets.
- LOAD_PREPARED()
- Chains everything. engineer(load_raw()) then build_matrix(). So my other scripts call one function and cannot run them out of order.

SRC/TRAIN.PY

- RANDOM_STATE = 42 everywhere so my results are reproducible.
- EVALUATE()
- predict_proba()[:, 1] to get churn probability, not hard 0/1 predictions. I need probabilities because my threshold is tuned separately later.
- roc_auc, pr_auc, f1, brier logged for every model.

- PR AUC is my main metric. My data is 26.5% churn so ROC AUC is optimistic under imbalance. PR AUC ignores true negatives.
- Brier score measures calibration. Whether a predicted 70% actually means 70%. AUC only cares about ranking so this is a separate property.
- BEST_F1_THRESHOLD()
- Computing F1 at every cutoff on the precision recall curve.
- np.clip to stop divide by zero when precision and recall are both 0.
- f1[:-1] because precision_recall_curve returns one more precision/recall pair than thresholds. Without the slice it is an index error.
- MODEL 1 - LOGISTIC REGRESSION
- Wrapped in a Pipeline with StandardScaler so scaling is fitted on train only. Doing it manually is how test set leakage happens.
- Scaling needed here because TotalCharges is in thousands and my flags are 0/1. Trees do not care, which is why models 2 and 3 skip it.
- class_weight="balanced" so it does not just predict "no churn" for everyone.
- MODEL 2 - RANDOM FOREST
- 400 trees, min_samples_leaf=5 to stop it memorising individual customers.
- n_jobs=-1 to use all cores.
- MODEL 3 - XGBOOST
- scale_pos_weight = ratio of non churners to churners, about 2.77. XGBoost version of class balancing.
- RandomizedSearchCV with n_iter=30. My grid has 2916 combinations, random search samples 30. Finds comparable optima much faster because most hyperparameters do not matter much.
- scoring="average_precision" so I tune on the same metric I report.
- cv=4 so each candidate trains 4 times. 120 fits total.
- After fitting I find the best F1 threshold and re-evaluate at it.
- NOTE: threshold is tuned on the test set which is mild leakage. Strictly it should come from a validation split. Documented not hidden.
- MLFLOW
- set_experiment("churn-prediction") groups all my runs.
- start_run() as a context manager so it closes even if the code raises.
- Logging params and metrics for all 3 models, model artifact for XGBoost only.
- SAVING
- joblib.dump saves model + threshold + column order together. Column order matters. Wrong order at prediction time gives silently wrong answers, not an error.
- MY RESULTS
- XGBoost tuned: ROC 0.846, PR 0.661, F1 0.642, Brier 0.163
- Random forest: ROC 0.845, PR 0.654, F1 0.637, Brier 0.148
- Logistic regression: ROC 0.844, PR 0.650, F1 0.620, Brier 0.166
- Tuned threshold 0.605

- XGBoost only beat logistic regression by 0.011 PR AUC. Small enough that I would consider shipping logistic regression. It trains in a second and its coefficients are readable without SHAP.
- Random forest has the best Brier so it is the best calibrated, which matters because my whole threshold analysis runs on probabilities.

SRC/EXPLAIN.PY

- PRETTY dict maps my engineered column names to readable labels for output.
- matplotlib.use("Agg") = non interactive backend. Without it matplotlib tries to open a window and crashes headless.
- Loading my saved model bundle and rebuilding the same split with the same seed.
- SHAP
- TreeExplainer is exact and fast for tree models.
- shap_values shape is (n_samples, n_features). One contribution per feature per customer, in log odds. Positive pushes toward churn.
- Comes from game theory. Shapley values were for splitting a payout fairly among players who contributed different amounts. Contributions sum to the prediction.
- GLOBAL
- np.abs() before mean. Without absolute a feature that pushes some customers up and others down averages near zero and disappears.
- Saving top 20 to global_importance.csv and a summary plot.
- MY TOP DRIVERS
- is_month_to_month 0.613
- tenure 0.394
- InternetService_Fiber_optic 0.277
- Contract_Two_year 0.191
- is_electronic_check 0.161
- avg_monthly_spend 0.147
- Contract type dominates, nearly 1.5x the next feature. Fibre optic churns more than DSL - that is a product question for the business, not a modelling one.
- LOCAL
- argsort()[::-1][:5] to get my 5 highest risk customers.
- For each one, sorting features by absolute SHAP and keeping top 4.
- Absolute again because "your two year contract is reducing your risk" is useful information for a retention agent.
- direction translated to "increases risk" / "reduces risk". Agent does not need the log odds number.
- Output to JSON not a chart, so it can join onto a CRM record.
- All 5 of my highest risk customers actually churned. Top one at 95.6% - 1 month tenure, month to month, fibre optic.

SRC/BUSINESS_IMPACT.PY

- This is the layer most churn projects skip. Model quality does not tell you where to set the cutoff. Cost does.
- MY ASSUMPTIONS (placeholders, meant to be replaced with real business figures)
- OFFER_COST = $40 per customer contacted
- CUSTOMER_VALUE = $500 expected value of a retained customer
- OFFER_SUCCESS = 30% of genuinely at risk customers the offer saves
- Sweeping threshold from 0.10 to 0.95 in steps of 0.05.
- confusion_matrix().ravel() gives tn, fp, fn, tp in that exact order. Wrong order silently inverts the whole analysis.
- contacted = tp + fp. I pay for everyone contacted including false positives.
- saved = tp * OFFER_SUCCESS. I only earn from true positives the offer converts.
- net = (saved * CUSTOMER_VALUE) - (contacted * OFFER_COST)
- MY RESULT
- Best net value at threshold 0.60.
- Curve is flat between 0.45 and 0.65. So the retention team can move the cutoff inside that band for operational reasons without losing money. More useful than a single optimal number.
- This is why the optimum is not the highest recall threshold. Contacting everyone catches every churner and loses money on offers to people who were staying.

SQL/FEATURES.SQL

- Mirrors my Python feature engineering as the warehouse query.
- In production the model reads from a warehouse not a CSV. Training and serving features have to match or I get training serving skew.
- CTEs (base, services, engineered) so it reads top to bottom instead of nesting.
- Conditional aggregation with MAX(CASE WHEN...) to pivot one row per customer per service into one row per customer with a column per service.
- Same divide by zero guard as my np.where in Python. Deliberately identical logic.
- Snowflake / Postgres compatible.

TESTS/TEST_FEATURES.PY

- sys.path.insert so tests can import features without a package install.
- @pytest.fixture(scope="module") loads the CSV once for all 8 tests instead of 8 times.
- MY 8 TESTS - all targeting failures that are silent, not loud
- total_charges_is_numeric - catches the text to number conversion failing
- zero_tenure_customers_have_zero_total_charges - catches wrong imputation
- no_internet_service_collapsed - catches the category collapse being skipped
- avg_monthly_spend_handles_zero_tenure - catches divide by zero producing NaN or inf
- num_addons_within_range - catches the count going outside 0 to 6

- matrix_has_no_target_leakage - my most important test. Catches the worst possible bug where the model sees the answer.
- matrix_is_fully_numeric - catches text columns surviving encoding
- column_names_are_model_safe - catches names XGBoost would reject
- All 8 passing.

SUPPORTING FILES

- requirements.txt - minimum versions with >=. Pinned exactly would be right for a deployed service, loose is right for a repo someone else will install.
- Makefile - make train / explain / impact / test. Encodes the run order in one place. train must run first because the other two load the model it saves.
- .gitignore - excluding **pycache**, .venv, mlruns, .DS_Store, model.joblib. Rule is git tracks what I wrote, not what my machine generated. NOT excluding reports/*.png because my README links them.
- .github/workflows/ci.yml - runs my tests on every push.

MY LAYERS

- 1 DATA - raw CSV in, clean dataframe out
- 2 FEATURES - clean data in, model ready matrix out. Where domain knowledge enters.
- 3 MODEL - matrix in, trained model and probabilities out
- 4 DECISION - probabilities in, actions out. Splits into SHAP explanations and threshold economics.
- Most projects stop at 3. Layer 4 is where a prediction becomes something a person can act on. The literature calls this the actionability gap.

WHAT I WOULD DO DIFFERENTLY

- Calibration. Random forest has a better Brier than my chosen XGBoost. Every downstream decision runs on probabilities so I would fit isotonic or Platt calibration and recheck whether the PR AUC advantage survives.
- Temporal validation. My split is random but churn data has time structure. A proper evaluation trains on an earlier period and tests on a later one. I expect the numbers would drop.
- Drift monitoring. Nothing here watches for feature drift after deployment, which is where churn models decay first. A pricing change shifts charge_delta and the model degrades silently.